CRIPTA — Documento de Decisiones de Diseño

Oct 16, 2026 · @Cesar

CRIPTA es un juego de exploración de criptas por eventos: no hay rondas, solo un reloj virtual que ejecuta lo que toque en cada instante, y este documento explica por qué cada estructura de datos que lo sostiene fue una buena elección y qué otra habría funcionado.

Cómo leer este documento

Cada sección cubre un problema del §4 del enunciado y sigue el orden que pide el §8: el problema con las operaciones que el juego realmente ejecuta, la decisión, su complejidad, una alternativa que también habría servido con un pro y un contra de cada lado, y los límites donde la decisión dejaría de ser apropiada. Cuando varias piezas del código usan la misma estructura (por ejemplo, tres listas enlazadas), se agrupan y se discuten una sola vez.

Las restricciones que moldearon todas las decisiones vienen del enunciado:

- **§7:** ningún `dict` como índice o almacenamiento del modelo; `list` solo como almacenamiento contiguo de bajo nivel, y el resto con estructuras propias.
- **§2.3 y §4.1:** el tiempo es un entero virtual, no el reloj de pared, y el orden de ejecución debe ser determinista (mismo tiempo, desempata el número de secuencia).
- **§4.9:** nada de `list.sort()` ni `sorted()` en los ordenamientos evaluados.
- **§5 y §9:** el presupuesto de solicitudes de red es parte del contrato, y la defensa será con una cripta que no vimos, posiblemente mucho mayor.

Estado de la evidencia empírica

El §8 exige mediciones propias obtenidas con `--bench` y medir también la alternativa descartada. La bandera `--bench` todavía no está implementada, así que **este documento no incluye cifras medidas**: en cada sección se indica qué se mediría y contra qué alternativa. Las complejidades que aparecen son análisis, no resultados experimentales, y se deben contrastar antes de la defensa.

1. Agenda de eventos del reloj virtual

La agenda es un **montículo binario (heap) de mínimos escrito a mano**, ordenado por (tiempo de ejecución, número de secuencia), donde cada evento recuerda su posición dentro del arreglo.

Problema. Todo lo que ocurre en CRIPTA es un evento en el futuro: cada acción de un enemigo, el cierre automático de una puerta, el rearme de una trampa, el fin de una poción o de una antorcha. El juego repite un ciclo sin parar: mirar cuál es el evento más próximo, sacarlo, ejecutarlo y, casi siempre, programar otro. Además hay dos operaciones menos frecuentes pero obligatorias: **cancelar** un evento que ya estaba programado (el pergamino deshace una puerta que iba a cerrarse sola, un enemigo muere con una acción pendiente) y **reprogramar** el tiempo restante de las acciones de un actor cuando su velocidad cambia (§2.3). Con varios cientos de eventos pendientes, la operación que se repite decenas de miles de veces es «dame el más próximo», y hacerla mal (recorrer todo en cada paso) multiplica el costo de cada ciclo por el tamaño de la agenda.

Por qué fue buena decisión.

- Insertar y sacar el mínimo cuestan O(log n); consultar el siguiente sin sacarlo cuesta O(1) (`heap[0]`), que es lo que necesita el ciclo para decidir si ya toca el turno del jugador.
- El desempate (§4.1) queda dentro de la propia comparación: a igual tiempo gana el número de secuencia más bajo, así que dos corridas con la misma semilla ejecutan los eventos en el mismo orden, requisito del `--replay`.
- Cada evento guarda su posición en el arreglo, lo que permite **quitar un evento cualquiera en O(log n)** sin recorrer la agenda. Sin ese dato, cancelar una puerta o reprogramar una acción costaría O(n) cada vez.
- Reprogramar no modifica el evento: lo reemplaza por uno nuevo con número de secuencia nuevo, como pide §2.3, y ambos movimientos quedan en el historial de retroceso (sección 3), así que deshacerlo es mover los mismos objetos en sentido contrario.

Complejidad.

Operación	Costo	Frecuencia en el juego
Programar un evento	O(log n)	Una o más por cada acción

Operación	Costo	Frecuencia en el juego
Sacar el siguiente	O(log n)	Una por cada evento ejecutado
Ver el siguiente	O(1)	Una por evento, para decidir el turno
Cancelar un evento concreto	O(log n)	Ocasional: puertas, muertes, pergamino
Reajustar velocidad de un actor	O(n)	Solo al empezar o terminar un efecto de velocidad

El reajuste de velocidad es lineal a propósito: ocurre dos veces por poción, no por ciclo, y mantener un índice por actor para evitarlo habría añadido una segunda estructura que sincronizar sin ganar nada medible.

Alternativa que también habría funcionado: lista ordenada con inserción por búsqueda binaria.

	Lista ordenada (arreglo contiguo)	Heap con posición (elegido)
Pro	Ver y sacar el siguiente es O(1) si se mantiene en orden inverso; el código es más corto y la agenda queda siempre ordenada, lo que facilita depurar y mostrarla	Insertar y cancelar son O(log n) incluso con muchos eventos simultáneos
Contra	Cada inserción o cancelación desplaza elementos: O(n). Con varios cientos de eventos y criptas más grandes, el costo crece justo en la operación frecuente	Solo el mínimo está ordenado; para ver toda la agenda hay que extraer o copiar y ordenar, y hay que mantener el campo `posicion` sincronizado en cada intercambio

Para una cripta chica la lista ordenada habría dado resultados equivalentes y más simples; el heap se justifica por el requisito de la defensa (§9), que apunta a una cripta con más actores y efectos simultáneos.

Límites conocidos. Si la agenda nunca superara unas decenas de eventos, la diferencia con la lista ordenada sería imperceptible y el heap sobraría. Si se necesitara consultar la agenda por actor con frecuencia (por ejemplo, para mostrar «próxima acción de cada

enemigo»), el barrido lineal por actor dejaría de ser aceptable y habría que añadir un índice por actor.

Medición pendiente (`--bench`). Insertar y sacar N eventos con N = 100, 1 000 y 10 000 en el heap y en la lista ordenada, y cancelar la mitad de ellos; sirve también para comprobar el requisito de §5.2 de menos de 20 ms por ciclo entre dos decisiones del jugador.

2. Caché de fichas y almacenamiento en disco

Las fichas del catálogo viven en dos niveles: una **caché LRU de capacidad fija** (una cola doble de entradas `[clave, valor, veces_en_uso]`, las más recientes al frente) y, debajo, un **archivo binario indexado en disco** (encabezado, bloques de datos, índice y pie con la posición del índice).

Problema. Cada solicitud HTTP cuenta contra el presupuesto de la cripta y además tarda del orden de 400 ms (§5). El juego necesita las mismas fichas una y otra vez: un enemigo, su arma, el objeto que suelta. La caché recibe muchas más búsquedas que inserciones, y debe cumplir tres cosas a la vez: no pasar de su capacidad, no desalojar una ficha que una sala cargada sigue usando, y no perder el trabajo hecho entre ejecuciones (el disco evita volver a gastar solicitudes).

Por qué fue buena decisión.

- **LRU:** lo que el jugador usó hace poco es lo que más probablemente vuelva a necesitar (las salas vecinas, los enemigos que lo persiguen). Desalojar lo que lleva más tiempo sin tocarse mantiene calientes las fichas de la zona actual.
- **Contador «en uso»:** una ficha que pertenece a un objeto o enemigo vivo en una sala cargada nunca se desaloja. Si todas están en uso, la nueva no se guarda (la capacidad jamás se supera) y el dato igual queda en disco. Esto evita que desalojar una ficha obligue a repetir una solicitud mientras se juega.
- **Contadores de aciertos y fallos:** salen gratis de la propia búsqueda y alimentan el reporte final del §5.1 («una caché caliente debería permitir usar menos solicitudes»).
- **Cola doble en vez de arreglo:** mover una entrada al frente al usarla no desplaza el resto del arreglo.
- **Disco con índice al final:** el archivo se escribe a un temporal y se reemplaza al terminar, así que un corte a la mitad no deja un archivo roto; el encabezado valida el formato y la versión; y una ficha o sala se lee posicionándose directo en su bloque sin leer el resto.

Complejidad. La búsqueda en la caché es lineal en su capacidad, O(c); con la capacidad por defecto (25) y la bandera `--cache-size` para ajustarla, c es pequeña y constante, y por eso la Búsqueda secuencial es aceptable aquí. En disco, localizar una ficha recorre el índice (O(k) por k claves del archivo) y luego lee un solo bloque; escribir reescribe el archivo completo, O(tamaño), lo que es aceptable porque las escrituras ocurren al traer lotes nuevos de la red, no en cada ciclo del juego.

Alternativa que también habría funcionado: tabla hash propia más lista doblemente enlazada.

	Tabla hash propia + lista doble (alternativa)	Cola doble con búsqueda lineal (elegido)
Pro	Búsqueda, inserción y desalojo O(1) en promedio, sin importar la capacidad; escala si la caché se agranda mucho	Código corto y fácil de razonar; sin función de dispersión ni colisiones; con capacidades chicas el costo es despreciable
Contra	Hay que escribir y probar la tabla (§7 prohibe `dict` en el modelo), sincronizarla con la lista y manejar colisiones y redimensionado	Cada búsqueda recorre hasta c entradas; si la capacidad creciera a miles, la operación más frecuente se volvería la más cara

Si la defensa usara una cripta mucho mayor y se pidiera una caché grande, esta es la primera pieza que habría que cambiar.

Límites conocidos.

- Con una capacidad de miles de fichas, el barrido lineal de la caché dejaría de ser despreciable.
- El archivo indexado relee el índice completo en cada consulta y reescribe todo en cada escritura; con cientos de salas guardadas, escribir lote por lote sería cada vez más caro. La salida es dividir por cripta (ya se hace) y, si hiciera falta, añadir al final en vez de reescribir.
- Los valores se guardan como JSON; es un formato sencillo y depurable a costa de tamaño en disco.

Medición pendiente (`--bench`). Tiempo de búsqueda con aciertos y con fallos en capacidades de 10, 25, 100 y 1 000, contra una implementación con tabla hash propia; tasa de aciertos de una partida completa con distintas capacidades; y costo de recuperar desde disco una ficha liberada.

3. Historial de retroceso (pergamino)

El historial es una **lista doblemente enlazada de nodos de cambio que se usa como pila**: cada nodo guarda uno o más cambios (el valor anterior y el nuevo de lo que se modificó), y un nodo especial marca dónde empieza cada acción del jugador. Solo se conservan las últimas 5 acciones.

Problema. El pergamino de retroceso debe devolver el juego al instante anterior a la última acción del jugador, y eso incluye lo que pasó por consecuencia: golpes de enemigos, una trampa que se activó, una puerta que se programa para cerrarse, el reloj y hasta eventos encolados o desencolados en la agenda. Cada ataque, recolección, movimiento, cambio de velocidad o evento de puerta deja una huella distinta, y la operación que se repite es **agregar al final** (cada vez que algo cambia) y **quitar del final** (al deshacer), mientras que descartar la acción más antigua ocurre solo cuando se pasa del límite.

Por qué fue buena decisión.

- **Guardar diferencias en vez de copias:** cada cambio se registra como «atributo X pasó de a a b». Registrar un golpe cuesta unos pocos campos, no una copia del mundo; el costo de guardar crece con lo que realmente cambió, no con el tamaño de la cripta.
- **Pila sobre lista doble:** agregar al final y sacar del final son O(1), sin desplazar nada, y la lista no tiene un tamaño máximo que reservar de antemano ni que redimensionar.
- **Marcadores de acción:** permiten deshacer una acción completa (la del jugador y todo lo que provocó hasta la siguiente decisión) retirando nodos hasta encontrar el marcador, sin buscar ni contar.
- **Límite de 5 acciones:** acota la memoria sin importar cuánto dure la partida. Descartar la más antigua solo recorre los nodos de esa acción, no todo el historial.
- **Acción rechazada:** como el marcador se abre antes de ejecutar, una acción que el juego rechaza se descarta sin gastar un lugar del historial ni mover el reloj (§2.13), y el recorte se difiere hasta que la acción se acepta para no perder una acción válida anterior.

Complejidad.

Operación	Costo
Registrar un cambio	O(1)
Deshacer la última acción	O(k), con k = cambios de esa acción
Descartar la acción más antigua	$O(k_a)$, con k_a = cambios de esa acción

Operación	Costo
Memoria total	Acotada por 5 acciones, sin importar la duración de la partida

Alternativa que también habría funcionado: política de instantáneas (copia del estado antes de cada acción) en un buffer circular de 5.

	Instantáneas en buffer circular (alternativa)	Diferencias en lista doble (elegido)
Pro	Deshacer es copiar de vuelta: no hay que escribir código de reversión por cada tipo de acción, así que olvidar un caso es imposible	La memoria y el tiempo de registro dependen de lo que cambia, no del tamaño del mundo
Contra	Cada acción copia todo el estado (salas cargadas, inventario, agenda): costo proporcional al mundo en cada paso, y copiar la agenda de eventos conserva objetos duplicados	Cada tipo de cambio necesita su propio código de reversión; si se agrega un efecto nuevo y se olvida registrarlo, el pergamino lo deja sin deshacer

Para un mundo pequeño las instantáneas habrían sido más seguras; las diferencias ganan cuando la cripta tiene muchas salas cargadas, que es el caso de la defensa.

Límites conocidos.

- El mayor riesgo no es el rendimiento sino la **completitud**: cada mecánica nueva (por ejemplo, el veneno y la regeneración del §2.10, todavía sin implementar) debe registrar sus cambios o el retroceso quedará inconsistente. Por eso las pruebas de retroceso comparan una foto completa del estado antes y después.
- El historial es un único objeto compartido por partida (patrón singleton); no admite dos partidas simultáneas en el mismo proceso.
- El historial no se guarda en disco; una partida cargada empieza con el historial vacío.

Medición pendiente (`--bench`). Costo de registrar 10 000 cambios y de deshacer acciones con 1, 10 y 100 cambios cada una; y la misma operación con la alternativa de instantáneas sobre una cripta de 50 salas cargadas, para medir cuánto cuesta copiar el mundo en cada acción.

4. Inventario del jugador

El inventario es una **lista doblemente enlazada con un cursor** (`actual`) que apunta al objeto seleccionado, con capacidad máxima fija. Los dos primeros nodos son los objetos equipados (arma y armadura).

Problema. El jugador recorre su mochila con «siguiente» y «anterior», usa o suelta el objeto bajo el cursor, recoge objetos al final y equipa o desequipa moviendo un objeto a los primeros lugares. El pergamino de retroceso, además, debe poder devolver un objeto a la **posición exacta** donde estaba. Las operaciones que se repiten son mover el cursor, agregar al final y quitar el objeto seleccionado; buscar un objeto por identidad y reinsertar en una posición dada ocurren solo al usar, equipar o deshacer.

Por qué fue buena decisión.

- Mover el cursor al vecino, agregar al final y desenlazar el nodo seleccionado son O(1) y no desplazan al resto, que es el patrón de uso real de una mochila.
- Quitar un objeto en medio no deja huecos que compactar ni obliga a renumerar índices.
- La capacidad se controla con un contador, sin reservar ni redimensionar memoria; el límite de peso o de espacio es una simple comparación.
- Los objetos equipados son simplemente los dos primeros nodos, de modo que «equipar» es reenlazar un nodo y «deshacer» es reenlazarlo en su índice anterior, sin una estructura extra que mantener sincronizada.
- Las vistas ordenadas (por peso, valor o nombre) no tocan la lista: copian los objetos a una lista de trabajo y la ordenan ahí, así el orden «real» de la mochila, que importa para los equipados y para el retroceso, nunca cambia.

Complejidad.

Operación	Costo	Frecuencia
Mover el cursor	O(1)	Muy frecuente
Agregar al final	O(1)	Frecuente
Soltar o usar el objeto del cursor	O(1)	Frecuente
Buscar un objeto por identidad	O(n)	Al usar, equipar o deshacer

Operación	Costo	Frecuencia
Reinsertar en un índice (retroceso)	O(n)	Solo al usar el pergamino
Vista ordenada	O(n) para copiar + costo del ordenamiento	Al consultar la mochila

n es la capacidad del inventario, que es chica y fija; por eso los recorridos lineales son aceptables.

Alternativa que también habría funcionado: arreglo contiguo con índice de cursor.

	Arreglo con índice (alternativa)	Lista doble con cursor (elegido)
Pro	Acceso por índice O(1) y buena localidad en memoria; reinsertar en una posición para el retroceso es un solo código; las vistas ordenadas se trabajan directo sobre el arreglo	Quitar y reenlazar un nodo en medio no desplaza elementos; el cursor siempre apunta a un nodo válido sin recalcular índices
Contra	Quitar en medio desplaza todos los elementos siguientes, O(n); hay que ajustar el índice del cursor tras cada quita; el espacio se reserva por adelantado	Sin acceso por posición: ir al índice i recorre i nodos; cada nodo agrega dos referencias de memoria

Con una capacidad de unas pocas decenas de objetos, ambas habrían sido igual de rápidas en la práctica; la lista gana por claridad en las operaciones que modifican el medio y porque el cursor y los equipados se expresan sin índices.

Límites conocidos. Si la capacidad del inventario creciera a cientos o miles de objetos, las búsquedas lineales por identidad y las reinserciones del retroceso dejarían de ser despreciables y habría que añadir un índice auxiliar. Además, el módulo del inventario importa hoy el ordenamiento desde la carpeta del controlador, una dependencia que habría que mover a un módulo neutral para respetar la separación del MVC del §6.

Medición pendiente (`--bench`). Tiempo de agregar, quitar el objeto del cursor y reinsertar por índice con capacidades de 10, 100 y 1 000, comparado con un arreglo contiguo con índice.

5. Ordenamiento

Se implementaron **dos algoritmos propios y estables, ordenamiento por inserción y ordenamiento por mezcla (merge sort)**, y una función que elige entre ellos: hasta 15 elementos usa inserción, con más usa mezcla. Ambos reciben una función de clave, así que sirven para cualquier criterio sin conocer el tipo de dato.

Problema. El §4.9 pide ordenar en contextos distintos: las vistas del inventario (por peso, valor o nombre, sobre listas de pocas decenas de objetos), la tabla de puntajes (una lista que crece partida tras partida, con clave compuesta de varios campos) y listas auxiliares. Los tamaños, el orden previo de los datos y el costo de comparar cambian de un caso a otro, y por eso el enunciado exige dos algoritmos con características distintas y una elección automática. Prohibe `list.sort()` y `sorted()`.

Por qué fue buena decisión.

- **Inserción:** en listas pequeñas o casi ordenadas hace muy pocas comparaciones y mueve elementos sin memoria extra. Es el caso de una mochila que ya estaba ordenada y recibió un objeto nuevo.
- **Mezcla:** garantiza O(n log n) sin importar cómo lleguen los datos, que es lo que se necesita para una tabla de puntajes que crece. Es estable, así que ordenar por varios criterios sucesivos conserva el orden anterior entre iguales.
- **Estabilidad en ambos:** el resultado es determinista para empates, y eso importa para que dos corridas con los mismos datos muestren la misma tabla.
- **La función de clave:** extrae el dato a comparar una sola vez por criterio de uso y permite que la lista enlazada del inventario no sepa nada del ordenamiento, que recibe una copia en lista.

Complejidad.

Algoritmo	Mejor caso	Peor caso	Memoria extra	Cuándo conviene
Inserción	O(n), datos ya ordenados	O(n²)	O(1)	Pocos elementos o casi ordenados
Mezcla	O(n log n)	O(n log n)	O(n)	Muchos elementos, orden desconocido

El umbral de 15 elementos es el punto donde el costo cuadrático de la inserción todavía es menor que el de crear las sublistas de la mezcla. **Ese número es una estimación inicial que debe confirmarse con `--bench`**, no un resultado medido.

Alternativa que también habría funcionado: un solo algoritmo, ordenamiento rápido (quicksort) con pivote mediana de tres, o montículo (heapsort).

	Un algoritmo único (alternativa)	Inserción + mezcla según tamaño (elegido)
Pro	Menos código y una sola estrategia que probar y explicar; quicksort ordena en el lugar y es muy rápido en promedio	Cada algoritmo trabaja en el rango donde es bueno, y el requisito del §4.9 de dos algoritmos con características distintas se cumple de forma natural
Contra	Quicksort no es estable y puede degradarse a $O(n^2)$ con ciertos datos; heapsort es estable solo con trabajo extra y tiene mala localidad; ninguno aprovecha que los datos ya vengan ordenados	Hay dos caminos que probar; la elección por tamaño es una regla simple que ignora si los datos ya están casi ordenados o si comparar es caro

Límites conocidos.

- La regla actual **solo mira el tamaño**. El enunciado pide decidir por propiedades medibles de los datos (tamaño, grado de orden previo y costo de la comparación). Una mejora razonable es medir el número de pares ya en orden en una pasada barata y, si los datos están casi ordenados, usar inserción incluso con listas grandes.
- La mezcla copia sublistas en cada nivel; con listas de decenas de miles de elementos, esa memoria extra y las copias podrían notarse frente a una mezcla que use un solo arreglo auxiliar.
- El ordenamiento hoy se usa en el inventario y en la tabla de puntajes; los índices del archivo de guardado y las listas auxiliares del catálogo (§4.9) todavía no pasan por él.

Medición pendiente (`--bench`). Ambos algoritmos con N = 5, 15, 50, 500 y 5 000 sobre tres tipos de entrada (aleatoria, casi ordenada, inversa) y con comparación barata y cara (clave compuesta de cuatro campos); de ahí sale el umbral real y se confirma o se corrige el 15.

6. Guardado binario de la partida

La partida se guarda en **un archivo binario de formato propio**: encabezado con marca y versión, bloques de datos (uno por sección del estado y uno por sala), un índice de secciones, una **tabla de salas ordenada por número** (posición, largo y CRC de cada bloque) y un pie de tamaño fijo que apunta a todo lo anterior. Los valores se codifican con una marca de un byte por tipo y enteros de largo variable (7 bits por byte, con signo en zigzag), sin `pickle`, sin `json` y sin diccionarios.

Problema. El §4.10 pide tres cosas a la vez: formato propio, **recuperar una sala específica sin leer el archivo completo** y detectar un archivo de otra versión del programa; y además comparar tamaño y tiempo de carga contra guardar lo mismo en JSON. Una partida guarda las salas cargadas con sus enemigos, trampas, salidas y objetos, el inventario, el reloj con su agenda, la bitácora y el contador de acciones; lo que cambia de una partida a otra es sobre todo la cantidad de salas.

Por qué fue buena decisión.

- **Pie fijo al final:** el lector salta directo al final, lo valida (marca de cierre y coherencia de posiciones) y desde ahí llega al índice sin recorrer nada. Un archivo cortado a la mitad se detecta de inmediato porque le falta la marca.
- **Tabla de salas ordenada con entradas de tamaño fijo:** como cada entrada ocupa lo mismo, se puede saltar a la i-ésima sin leer las anteriores, y como está ordenada se encuentra una sala con **búsqueda binaria en O(log s)** lecturas pequeñas, para luego leer solo ese bloque.
- **CRC por bloque:** una sala dañada se detecta al leerla, y no contamina el resto de la partida.
- **Enteros de largo variable:** los números chicos (casi todos: vida, ids, tiempos) ocupan un byte, lo que reduce el tamaño frente a un ancho fijo de ocho bytes o a texto.
- **Escritura a un temporal y reemplazo:** si el programa se cierra a mitad de guardar, el archivo anterior sigue intacto.
- **La versión del formato en el encabezado** hace que un archivo de otra versión se rechace con un mensaje claro en vez de leerse mal (por ejemplo, al sumar el contador de acciones del jugador se subió la versión del formato).
- **El modelo no conoce el archivo:** el estado se exporta como datos simples y es un adaptador el que los escribe, así que cambiar el formato no toca la lógica del juego.

Complejidad.

Operación	Costo
Guardar la partida	O(tamaño del estado), una sola pasada
Cargar una sala	O(log s) lecturas en la tabla + una lectura del bloque, con s salas guardadas
Cargar la partida completa	O(tamaño del estado)
Listar las salas guardadas	O(s), sin leer ningún bloque de sala

Alternativa que también habría funcionado: registros de tamaño fijo con struct, una fila por entidad.

	Registros fijos con `struct` (alternativa)	Codificación con marcas y enteros variables (elegido)
Pro	Leer cualquier entidad es calcular una posición: acceso O(1) sin tabla; formato muy simple de documentar y verificar	Compacto; los campos opcionales y las listas de largo variable no desperdician espacio; un tipo nuevo se agrega al final sin reescribir el resto
Contra	Los textos y listas de largo variable (nombres, inventario, mensajes) obligan a reservar un ancho máximo y desperdiciar espacio, o a mantener tablas auxiliares; agregar un campo cambia el tamaño de todos los registros	Para llegar a una sala hace falta la tabla; no se puede calcular su posición directamente porque los bloques miden distinto

La tabla de puntajes sí usa registros fijos (nombre de hasta 15 caracteres, texto de largo fijo para cripta y versión), porque allí todos los registros son del mismo tipo y agregar uno nuevo es escribir al final.

Límites conocidos.

- Guardar siempre escribe todo el estado; con cientos de salas cargadas, un guardado incremental (solo las salas que cambiaron) sería más rápido pero más complejo.
- Las secciones del estado se buscan en orden porque son pocas; si creciera su número, convendría indexarlas igual que las salas.
- El historial del pergamino **no** se guarda: una partida cargada empieza sin poder retroceder. Es una limitación aceptada y documentada.

- No hay compresión; para partidas muy grandes se podría añadir por bloque.

Medición pendiente (`--bench`). Tamaño en disco y tiempo de guardar y de cargar la misma partida en el formato binario y en JSON, con 10, 100 y 1 000 salas cargadas; y tiempo de cargar una sola sala frente a cargar todo, para demostrar el acceso directo. Esta es la comparación que el §4.10 pide explícitamente y que hoy no está hecha.

7. Bitácora de partida y reproducción (`--replay`)

Cada acción del jugador se guarda en dos lugares con roles distintos: en memoria, una **lista doblemente enlazada de acciones en orden cronológico** (`LogEventos`); y en disco, un **archivo de texto de solo anexado** (`AlmacenLogTexto`) con un encabezado de cuatro líneas (cripta, semilla, versión de cripta, versión de catálogo) y una línea por acción.

Problema. El §4.11 exige que cada acción quede registrada en un archivo de texto de solo anexado y que `--replay partida.log` reproduzca la partida completa sin interacción. Para que la reproducción sea fiel, el juego tiene que ser determinista (mismas acciones, misma semilla y mismos datos dan la misma partida) y, si la cripta o el catálogo cambiaron desde que se grabó, el programa debe reportar que no puede garantizar una reproducción fiel en lugar de reproducir en silencio contra datos distintos.

Por qué fue buena decisión.

- **Agregar al final en O(1) y recorrer en orden:** una acción nueva es enlazar un nodo, sin copiar ni desplazar lo anterior. El mismo orden sirve para escribir en disco y para reproducir.
- **Texto de solo anexado:** cada acción se escribe en el instante en que ocurre y el archivo se abre en modo exclusivo, así que ni un cierre inesperado ni un nombre repetido pueden perder o pisar un log. Es legible a simple vista y se puede revisar con cualquier editor.
- **Una línea por acción con los datos mínimos:** tipo de acción, tiempo, actor y referencias por identificador (nunca objetos completos). Una referencia ligera se resuelve contra el estado de la partida que se reproduce, así que el log no necesita crecer con el mundo.
- **Encabezado con versiones:** permite comprobar antes de empezar que la cripta y el catálogo siguen siendo los mismos que cuando se grabó.
- **Un solo generador de azar por partida:** la semilla del encabezado reconstruye la misma secuencia de tiradas, de modo que el orden en que se resuelven los eventos (garantizado por el desempate de la agenda) fija el resultado, y no el orden en que

lleguen respuestas de red.

- **El modelo no escribe archivos:** avisa cada acción a un registrador, y es el adaptador el que la guarda. Así se pudo separar la E/S de la lógica y reproducir sin crear un log nuevo.

Complejidad.

Operación	Costo
Registrar una acción (memoria)	O(1)
Anexar una línea al archivo	O(1), una escritura pequeña
Reproducir la partida	O(a), con a acciones, más el costo de cada ciclo de eventos
Leer y validar el encabezado	O(1)

Para interpretar cada línea (que es JSON) se usa el diccionario que produce `json.loads` solo de forma transitoria, para leer el texto; no se guarda ni se usa como índice, así que respeta el §7.

Alternativa que también habría funcionado: arreglo dinámico (`list`) en memoria y un formato binario para el log.

	Arreglo dinámico + log binario (alternativa)	Lista enlazada + log de texto (elegido)
Pro	El arreglo da acceso por índice O(1) y mejor localidad; un log binario es más compacto y más rápido de leer	Anexar nunca copia ni redimensiona; el log de texto se puede leer, revisar y editar a mano para armar escenarios de prueba, que es justo el uso de `--replay` en la revisión
Contra	Un arreglo que crece copia sus elementos cada cierto tiempo; un binario no se puede inspeccionar sin herramienta y es más difícil de depurar	Cada nodo agrega referencias de memoria; el texto ocupa más que el binario y cada línea hay que interpretarla al reproducir

Límites conocidos.

- Una partida de cientos de miles de acciones haría que la lista en memoria y el texto pesen más de lo óptimo; para el tamaño realista de una partida no es un problema.
- La reproducción es fiel solo mientras las versiones coincidan; con una versión distinta el programa se niega a reproducir, a propósito. No guardamos copias locales de versiones anteriores, así que no hay camino de recuperación más allá de reportarlo.
- Una acción rechazada por el juego no se registra (no consume tiempo ni modifica nada), así que el log contiene solo lo que efectivamente ocurrió.

Medición pendiente (`--bench`). Tiempo de registrar 10 000 acciones en la lista enlazada frente a un arreglo, tamaño del log de texto frente a un binario equivalente y tiempo de reproducir una partida de 1 000 acciones, para comprobar que no domina el costo de los ciclos de eventos.

8. El mundo: salas, vecindad, mensajes recientes y puntajes

Cuatro piezas más pequeñas completan el juego, y se agrupan por el tipo de estructura que comparten: un **arreglo indexado por número de sala**, una **cola** (usada tanto para recorrer vecinos como para la bitácora de mensajes) y una **tabla de registros fijos** para los puntajes.

Salas como arreglo indexado por número. La lista de salas del juego tiene un puesto por cada número de sala, y las salas que todavía no se cargaron quedan vacías. Encontrar la sala del jugador, la de un enemigo o la de una trampa es un acceso directo O(1), y es la consulta más frecuente del ciclo (cada acción y cada evento de enemigo la usa). Esto cumple el §7, porque se usa `list` como almacenamiento contiguo de bajo nivel y no como índice de un diccionario. **Alternativa:** una lista enlazada de salas cargadas. **Pro:** no reserva puestos para salas que nunca se visitan, útil si la cripta fuera enorme y casi toda quedara sin cargar. **Contra:** encontrar una sala pasa a ser O(s) y habría que recorrerla en cada evento. **Límite:** el arreglo asume que los números de sala son consecutivos desde 1; una cripta desconocida de la defensa con ids muy dispersos desperdiciaría espacio o fallaría, y habría que pasar a una estructura ordenada con búsqueda binaria.

Cola para recorrer vecinos y para los últimos mensajes. Las salas vecinas se encuentran con un recorrido por niveles sobre una cola de doble extremo, que visita primero todas las salas a un paso, luego a dos, y así hasta la profundidad pedida; el orden por distancia es lo que decide qué salas entran primero en un lote de contenido, que es una sola solicitud con hasta 10 salas. La bitácora de mensajes que ve el jugador es una **cola enlazada de 20 mensajes**: agregar uno nuevo es enlazarlo al final y, si se pasa de 20, soltar el del frente, ambas en O(1), sin desplazar los 19 anteriores (el requisito literal del §4.8). **Alternativa:** un

buffer circular sobre un arreglo de tamaño fijo. **Pro:** memoria reservada una sola vez y mejor localidad; no crea un nodo por mensaje. **Contra:** quitar un mensaje del medio (lo hace el retroceso al deshacer una acción) obliga a desplazar elementos, mientras que en la cola enlazada solo se reenlazan dos nodos. **Límite:** la comprobación de si una sala ya fue alcanzada recorre la lista de alcanzadas, O(n) por sala; con criptas de miles de salas conviene reemplazarla por una marca por número de sala.

Tabla de puntajes en registros fijos. Cada partida terminada agrega un registro de tamaño fijo al final del archivo (nombre de hasta 15 caracteres, cripta, versión, acciones, enemigos derrotados, tiempo final y si fue victoria). Agregar es O(1) y no reescribe el archivo, y leer todos es una pasada. La tabla se ordena en memoria con los algoritmos de la sección 5 y una clave compuesta: primero las victorias, luego menos tiempo, menos acciones y más enemigos derrotados. **Alternativa:** guardar los registros como texto (CSV o JSON). **Pro:** legible y editable a mano. **Contra:** hay que interpretar cada línea y un campo mal escrito rompe la lectura; los registros fijos se validan solos por tamaño. **Límite:** con muchos miles de partidas, ordenar en cada consulta empezaría a costar y convendría mantener el archivo ordenado o leer solo los mejores N.

Medición pendiente (`--bench`). Acceso a una sala por número en una cripta de 10, 100 y 1 000 salas frente a una lista enlazada; recorrido por niveles hasta profundidad 3 en una cripta ramificada; y ordenar una tabla de 1 000 y 10 000 puntajes con clave compuesta.

9. Qué se degrada primero si la cripta crece 50 veces

Las decisiones anteriores están pensadas para la cripta de referencia; si la de la defensa tiene 50 veces más salas y actores, lo primero en notarse no es la agenda de eventos (el heap escala bien) sino las búsquedas lineales sobre datos que crecen con la cripta. En orden de aparición esperado:

1. **Búsqueda de una sala en el esqueleto de la cripta.** Se hace recorriendo la lista de salas en cada consulta, y el recorrido por vecindad la repite para cada sala visitada. Con miles de salas, precargar vecinas se vuelve la parte más lenta. Solución: construir una vez un arreglo indexado por número de sala.
2. **Archivo indexado de fichas y criptas.** Cada lectura relee el índice y cada escritura reescribe el archivo entero. Solución: guardar el índice en memoria durante la sesión y añadir al final en lugar de reescribir.
3. **Caché con búsqueda lineal.** Aceptable con la capacidad actual; si la cripta grande exigiera una caché de cientos de fichas, es la operación con más búsquedas que inserciones y la primera candidata a pasar a una tabla hash propia.

4. **Comprobación de salas ya alcanzadas** en el recorrido por vecindad (O(n) por sala). Solución: una marca por número de sala.
5. **Reajuste de velocidad** (barrido por actor en la agenda): solo afecta cuando hay muchos actores y muchos cambios de velocidad seguidos; un índice por actor lo resolvería.
6. **Guardado completo:** crece con las salas cargadas, no con las existentes, así que se mantiene acotado por la precarga y por el tamaño de la caché.

Lo que **no** se degrada con el tamaño de la cripta: agregar y sacar eventos de la agenda (O(log n)), registrar y deshacer cambios (O(1) y acotado a 5 acciones), anexar a la bitácora y al log, el acceso a una sala ya cargada por número y la lectura de una sala del guardado (búsqueda binaria en la tabla).

10. Resumen de las decisiones

Problema	Estructura elegida	Alternativa considerada
Agenda de eventos	Heap binario con posición por evento	Lista ordenada con búsqueda binaria
Caché de fichas	Cola doble LRU con contador de uso + archivo indexado	Tabla hash propia + lista doble
Historial de retroceso	Lista doble como pila con marcadores de acción	Instantáneas en buffer circular
Inventario	Lista doble con cursor	Arreglo contiguo con índice
Ordenamiento	Inserción + mezcla según tamaño	Un solo algoritmo (quicksort o heapsort)
Guardado binario	Bloques con marcas + tabla de salas ordenada + pie fijo	Registros fijos con `struct`
Bitácora y reproducción	Lista doble en memoria + texto de solo anexado	Arreglo dinámico + log binario
Salas, mensajes y puntajes	Arreglo indexado, cola enlazada, registros fijos	Lista enlazada, buffer circular, texto (CSV/JSON)

11. Pendientes antes de la defensa

- Implementar `--bench` y rellenar la medición de cada sección, incluida la alternativa descartada; hasta entonces, las complejidades son análisis y no resultados.
- Implementar el veneno y la regeneración del §2.10, que cambian el contenido del historial de retroceso y de la agenda.
- Hacer que la selección del ordenamiento use más propiedades que el tamaño (§4.9) y extenderlo a los índices del guardado y a las listas del catálogo.
- Comparar el guardado binario con JSON en tamaño y tiempo de carga (§4.10).
- Mover el módulo de ordenamiento fuera de la carpeta del controlador, para que la lógica no dependa de él (§6).